Zewail City of Science, Technology and Innovation

University of Science and Technology

School of Computational Sciences and AI

CSAI 203 - Fall 2026

Introduction to Software Engineering

Campus Event & Student Club Hub

Phase 2: Software Requirements Specification (SRS)

Team Members:

Name	ID
Nada Mohammed Kulaib	202402221
Retaj Reda Elbaz	202400545
Shrouk Mohamed Mostafa	202402345
Salsabel AbdEl-Rahman	202401890

Representative Contact info: 202400545@zewailcity.edu.eg

Contents

1 Introduction **3**

1.1 Purpose 3
1.2 Scope 3
1.3 Definitions, Acronyms, and Abbreviations 3
1.4 References 3
1.5 Overview 3

2 Overall Description **4**

2.1 Product Perspective 4
2.2 Product Functions 4
2.3 User Classes and Characteristics 4
2.4 Software Stakeholders 4
2.5 Operating Environment 5
2.6 Design and Implementation Constraints 5
2.7 User Documentation 5
2.8 Assumptions and Dependencies 5

3 Specific Requirements **5**

3.1 Functional Requirements 5
3.1.1 FR-01: User Authentication & Role-Based Access Control . 6
3.1.2 FR-02: Student Profile Management 6
3.1.3 FR-03: Student Club Creation & Admin Approval 6
3.1.4 FR-04: Event Creation & Scheduling 6
3.1.5 FR-05: Event Browsing & Filtering 6
3.1.6 FR-06: Event Registration & Capacity Management 7
3.1.7 FR-07: Digital Ticket Generation (QR Code) 7
3.1.8 FR-08: Waitlist Management 7
3.1.9 FR-09: On-Site QR Ticket Scanning & Attendance Verification 7
3.1.10 FR-10: Manual Attendance Logging 7
3.1.11 FR-11: Event Cancellation & Automated Notifications 8
3.1.12 FR-12: Club Membership Application & Management 8
3.1.13 FR-13: Club Budget Allocation (Admin Feature) 8
3.1.14 FR-14: Expense Request Submission 8
3.1.15 FR-15: Expense Auditing & Approval 8

3.1.16 FR-16: Real-Time Financial Ledger & Analytics 9
3.1.17 FR-17: Personalized Student Dashboard 9
3.1.18 FR-18: Interactive FAQ Module 9
3.1.19 FR-19: Onboarding Tutorial Pop-up for New Club Leaders . 9
3.1.20 FR-20: Event Feedback & Rating System 9
3.2 Use Case Model 10
3.2.1 Use Case Diagrams 10
3.2.2 Use Case Descriptions (IEEE Template) 12
3.3 Domain Model 15
3.3.1 Conceptual Class Diagram 16
3.3.2 Class Descriptions 16
3.4 Non-Functional Requirements 17
3.4.1 NFR-01: Performance — Response Time 18
3.4.2 NFR-02: Security — Password Hashing & Role Protection . 18
3.4.3 NFR-03: Usability — Responsive Design & Accessibility . 18
3.4.4 NFR-04: Reliability — Data Consistency & Transaction Integrity 18
3.4.5 NFR-05: Maintainability — Architectural Conformance . . 18
3.5 External Interface Requirements 19
3.5.1 User Interface 19
3.5.2 Hardware Interface 19
3.5.3 Software Interface 19
3.5.4 Communication Interface 20

A Appendices **20**
A.1 Appendix A: Data Dictionary 20
A.2 Appendix B: Glossary 20

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to detail the requirements for the(Campus Event & Student Club Hub) platform. This document serves as a blueprint for the design and implementation phases of the software development life cycle. It will be used by the development team, university administrators, and stakeholders to understand the system's expected behavior, features, and constraints.

1.2 Scope

The Campus Event & Student Club Hub is a comprehensive web-based platform designed to centralize and streamline student organization activities. The system will enable student clubs to publish upcoming campus events, manage student registrations, and issue verifiable digital tickets. Furthermore, it provides tools for club leaders to track event attendance accurately and manage their allocated club budgets transparently. The platform aims to enhance student engagement and reduce the administrative burden on club organizers.

1.3 Definitions, Acronyms, and Abbreviations

- **SRS:** Software Requirements Specification.
- **Hub:** Refers to the Campus Event & Student Club Hub system.
- **Club Leader:** A student assigned administrative privileges over a specific student organization's page and budget.
- **UI:** User Interface.
- **MVC:** Model-View-Controller architecture.

1.4 References

- CSAI 203 Course Project Document, Fall 2026, Zewail City of Science and Technology.
- IEEE Guide for Software Requirements Specifications.

1.5 Overview

This document is organized into four main sections. Section 1 introduces the project. Section 2 provides an overall description of the system, including its functions, users, and constraints. Section 3 details the specific functional and non-functional requirements, use case models, domain models, and external interfaces. Finally, Section 4 contains the appendices, including the data dictionary and glossary.

2 Overall Description

2.1 Product Perspective

The Campus Event & Student Club Hub is a standalone web application built from scratch to serve the Zewail City university community. It will implement the Model-View-Controller (MVC) architectural pattern. While it operates independently, it is designed with the potential to integrate with the university's existing student email system for authentication and notifications.

2.2 Product Functions

The major functionalities of the system include:

- **Event Management:** Creating, updating, and publishing campus events with details (date, venue, capacity).
- **Registration & Ticketing:** Allowing students to register for events and generating digital tickets (e.g., QR codes).
- **Attendance Tracking:** Scanning or manually logging student attendance at events.
- **Budget Management:** Tracking club income, expenses, and current budget balances.
- **User Dashboard:** Providing personalized views for students (upcoming registered events) and club leaders (event analytics and financial summaries).

2.3 User Classes and Characteristics

- **Student (Standard User):** Requires minimal technical expertise. They use the platform to browse events, register, and access their digital tickets.
- **Club Leader (Organizer):** Requires moderate platform familiarity. They manage event postings, scan tickets, and input budget expenses.
- **System Admin (University Staff):** High technical or administrative authority. They approve new clubs, allocate initial budgets, and moderate content across the platform.

2.4 Software Stakeholders

- **Students:** Interested in easily finding and attending campus events.
- **Student Clubs/Organizations:** Interested in a streamlined way to reach audiences, manage attendees, and track their spending.
- **Student Affairs Department:** Interested in monitoring club activities, ensuring compliance with university policies, and auditing budgets.

- **Development Team:** Interested in delivering a functional, robust product that meets CSAI 203 project requirements.

2.5 Operating Environment

The system will operate as a responsive web application. It must be accessible via modern web browsers (Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge) on both desktop and mobile devices.

2.6 Design and Implementation Constraints

- **Architecture:** The system must follow the Model-View-Controller (MVC) architectural pattern, as required by the course project guidelines.
- **Data Integrity:** The system must ensure that event registrations, ticket records, attendance-dance records, and club budget transactions remain consistent and accurate.
- **Access Control:** The system must restrict access to administrative functions according to user roles. For example, only authorized club leaders should manage their club events and budgets, while system administrators should have access to system-wide management functions.

2.7 User Documentation

The system will provide an online FAQ section accessible to all users. Additionally, a brief(Getting Started) tutorial pop-up will be available for new Club Leaders to explain how to publish events and manage budgets.

2.8 Assumptions and Dependencies

- **Assumptions:** All target users possess a compatible device (smartphone or computer) with internet access.
- **Dependencies:** The system depends on reliable web hosting and database servers to ensuresure uptime and data integrity.

3 Specific Requirements

3.1 Functional Requirements

This section details the 20 functional requirement features of the Campus Event & Student Club Hub platform. Each requirement includes explicit operational scenarios covering front-end user interactions, back-end logic, and database state transitions.

3.1.1 FR-01: User Authentication & Role-Based Access Control

Description: The system shall allow users (Students, Club Leaders, and System Administrators) to log in securely using their university credentials. Upon authentication, the system must assign appropriate role-based permissions governing platform functionality.
Scenario: A student inputs their university email and password on the login page. The backend verifies the hashed credentials against the database. Upon success, the system creates a user session and redirects the user to the Student Dashboard. If a Club Leader logs in, the controller redirects them to the Club Management Panel with elevated administrative rights.

3.1.2 FR-02: Student Profile Management

Description: The system shall enable registered students to view and update their personal profile information, including profile picture, contact details, and club membership affiliations.
Scenario: A student navigates to their profile page and edits their bio and contact number. Upon clicking(Save Changes), the controller validates the inputs (e.g., verifying phone number formats) and updates the Model. The View updates instantly with a success message.

3.1.3 FR-03: Student Club Creation & Admin Approval

Description: The system shall allow students to submit formal requests to establish new student organizations, which must be reviewed and approved by System Administrators.
Scenario: A student fills out the(Request New Club) form with the proposed club name, mission statement, and faculty advisor details. The request status is set to(Pending). A System Administrator receives a notification in their dashboard, reviews the application, and clicks(Approve). The system then automatically provisions a new club page and elevates the requesting student to(Club Leader).

3.1.4 FR-04: Event Creation & Scheduling

Description: The system shall allow authorized Club Leaders to draft and publish new campus events by defining event title, description, category, date, time, venue, and maximum registration capacity.
Scenario: A Club Leader accesses the event management portal, fills out event specifications, uploads a poster image, sets the capacity to 150 students, and submits. The controller validates that the selected venue is not double-booked for that time slot. Upon validation, the event status changes to(Published) and appears in the public event feed.

3.1.5 FR-05: Event Browsing & Filtering

Description: The system shall provide a public feed where all authenticated users can browse, search, and filter upcoming and past campus events based on categories, dates, or organizing clubs.

Scenario: A student accesses the event discovery page and types(Robotics) into the search bar while selecting the filter(This Week). The controller queries the Model for active events matching these criteria and dynamically renders the filtered list on the View.

3.1.6 FR-06: Event Registration & Capacity Management

Description: The system shall permit authenticated students to register for published events, automatically tracking remaining seats and preventing overbooking.
Scenario: A student views an event page with 5 remaining seats and clicks(Register). The back-end initiates a transaction: it checks seat availability, decrements the available capacity to 4, records the registration in the database, and links it to the student's ID. If the capacity reaches zero, the registration button is disabled, and subsequent users are prompted to join a waitlist.

3.1.7 FR-07: Digital Ticket Generation (QR Code)

Description: Upon successful event registration, the system shall automatically generate a unique, tamper-proof digital ticket containing a scannable QR code for the student.
Scenario: Immediately after registering for an event, the system generates an encrypted string containing the registration ID and student ID, renders it as a QR code image, and displays it on the student's(My Tickets) page. An automated confirmation email with the attached ticket is also dispatched to the student's inbox.

3.1.8 FR-08: Waitlist Management

Description: The system shall maintain an automated waitlist for events that reach full capacity and automatically promote waitlisted users if a spot opens up.
Scenario: When an event is full, a student clicks(Join Waitlist). The system logs their position (e.g., 1 in line). If an already registered student cancels their reservation, the system automatically cancels their ticket, registers the first student on the waitlist, generates a new digital ticket for them, and sends a notification.

3.1.9 FR-09: On-Site QR Ticket Scanning & Attendance Verification

Description: The system shall provide Club Leaders with a mobile-friendly scanning interface to check in attendees at event entrances by scanning their digital QR tickets.
Scenario: At the venue entrance, a Club Leader uses their smartphone camera via the web app to scan a student's QR code. The controller decrypts the code, verifies its validity against the event registry, and marks the student's status as(Attended). The screen displays a green checkmark. If scanned twice, a red warning ()Ticket Already Used)) is shown.

3.1.10 FR-10: Manual Attendance Logging

Description: The system shall allow Club Leaders to manually search for students by ID or name and mark their attendance in case of technical issues or damaged screens.

Scenario: A student's phone battery dies at the door. The Club Leader switches to the(Manual Check-In) tab, searches the student's ID (202402221), verifies their identity, and clicks(Mark Present). The attendance log updates instantly in the database.

3.1.11 FR-11: Event Cancellation & Automated Notifications

Description: The system shall allow Club Leaders or Admins to cancel an event, automatically updating its status and broadcasting notifications to all registered attendees.
Scenario: A Club Leader clicks(Cancel Event) and inputs a reason (e.g.,(Bad Weather)). The system updates the event status to(Cancelled), invalidates all associated digital tickets, and sends an immediate email notification to all registered attendees explaining the cancellation.

3.1.12 FR-12: Club Membership Application & Management

Description: The system shall allow students to apply for general club memberships and enable Club Leaders to review, accept, or reject incoming membership applications.
Scenario: A student clicks(Join Club) on the Robotics Club page. The application goes to the club's pending requests queue. The Club Leader reviews the applicant's profile and clicks(Accept). The student receives a notification and gains access to exclusive member-only club announcements.

3.1.13 FR-13: Club Budget Allocation (Admin Feature)

Description: The system shall allow System Administrators (Student Affairs) to allocate and modify initial semester budgets for approved student clubs.
Scenario: An Admin opens the Financial Management module, selects the(IEEE Student Branch), enters an allocated budget of(15,000 EGP) for Fall 2026, and clicks(Save). The system creates a financial ledger entry and updates the club's available balance.

3.1.14 FR-14: Expense Request Submission

Description: The system shall enable Club Leaders to submit budget expense requests with supporting receipt images or invoices for financial tracking.
Scenario: A Club Leader purchases event decorations for 1,200 EGP. They open the Budget module, click(Log Expense), enter the amount, select the category (Event Supplies), upload a photo of the receipt, and submit. The transaction status is set to(Pending Audit).

3.1.15 FR-15: Expense Auditing & Approval

Description: The system shall allow System Administrators to review logged club expenses, inspect receipts, and approve or reject financial transactions.
Scenario: A Student Affairs staff member reviews pending expense logs. They click on the 1,200 EGP receipt, inspect the image, and click(Approve). The controller deducts 1,200 EGP from the club's active balance and updates the financial status to(Audited).

3.1.16 FR-16: Real-Time Financial Ledger & Analytics

Description: The system shall calculate and display real-time financial summaries, tracking allocated budgets, total approved expenses, and remaining balances for each club.
Scenario: A Club Leader opens their Financial Dashboard. The system aggregates all approved expense transactions, calculates the remaining balance, and renders a pie chart showing expenditure breakdowns by category (e.g., Catering, Marketing, Logistics).

3.1.17 FR-17: Personalized Student Dashboard

Description: The system shall provide students with a centralized dashboard displaying their upcoming registered events, active digital tickets, and club memberships.
Scenario: Upon logging in, a student lands on their dashboard. The top section displays cards for upcoming registered events with quick-access(View Ticket) buttons, while the side panel shows news feeds from clubs they have joined.

3.1.18 FR-18: Interactive FAQ Module

Description: The system shall provide a dedicated, searchable FAQ section accessible to all users to answer common questions regarding event booking, ticket verification, and club rules.
Scenario: A student types(How to cancel ticket) in the FAQ search bar. The system dynamically filters accordion elements, expanding the answer that guides the student through the ticket cancellation steps.

3.1.19 FR-19: Onboarding Tutorial Pop-up for New Club Leaders

Description: The system shall trigger an interactive(Getting Started) tutorial modal when a newly assigned Club Leader accesses the management portal for the first time.
Scenario:

3.1.20 FR-20: Event Feedback & Rating System

Description: The system shall enable verified attendees to submit ratings (1–5 stars) and textual feedback for events they attended after the event concludes.
Scenario: 2 hours after an event ends, the system sends an automated prompt to attendees who were scanned as(Present). A student opens the link, selects 5 stars, writes a brief comment (Great workshop!)), and submits. The event's aggregate score updates on the organizer's analytics panel.

3.2 Use Case Model

The Use Case model describes the proposed system's functional requirements from the end-users' perspective. It illustrates interactions between actors (Students, Club Leaders, System Administrators) and the primary system features.

3.2.1 Use Case Diagrams

This section contains three distinct Use Case diagrams categorizing system capabilities across different functional domains.

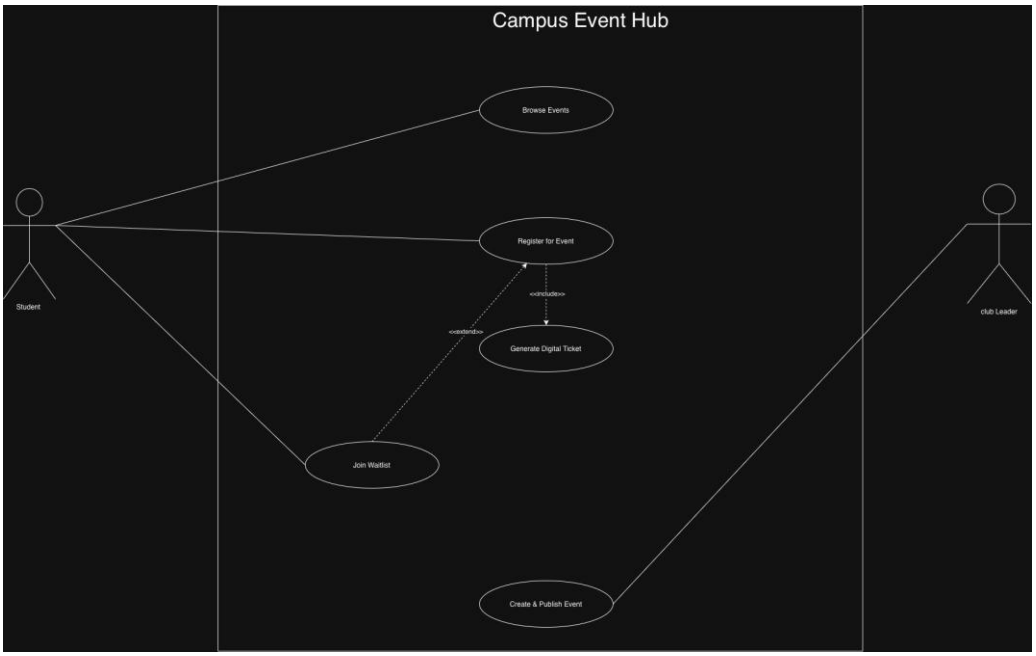


Figure 1: Use Case Diagram 1: Event Discovery, Registration, and Ticketing Core Functions.

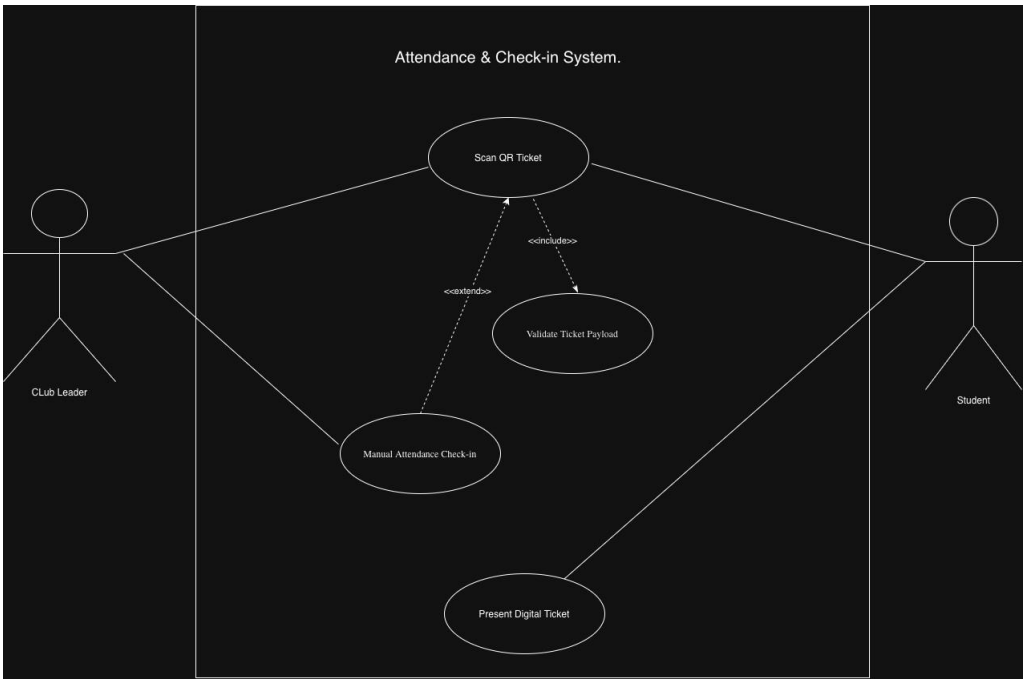


Figure 2: Use Case Diagram 2: On-Site QR Scanning and Attendance Verification.

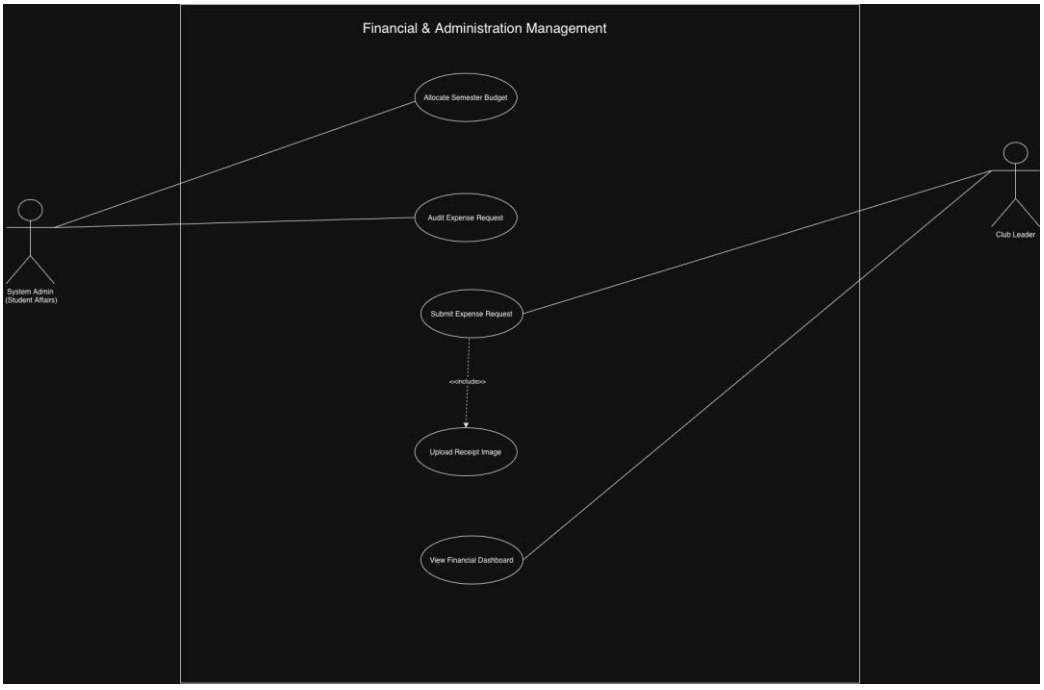


Figure 3: Use Case Diagram 3: Club Budget Allocation and Expense Auditing.

3.2.2 Use Case Descriptions (IEEE Template)

Below are the detailed use case descriptions formatted according to the exact template provided in the lecture slides.

UC-01: Register for Campus Event

- **Use Case ID:** UC-01
- **Use Case Name:** Register for Campus Event
- **Primary Actor:** Student
- **Stakeholders and Interests:**
  - *Student:* Wants to guarantee a seat at the event and receive a ticket quickly.
  - *Club Leader:* Wants an accurate headcount of attendees to manage venue capacity.
- **Preconditions:** The student is authenticated and logged in. The selected event is published and has available capacity (*Capacity* > 0).
- **Postconditions:** The student's registration is saved in the database, the event capacity is decremented by 1, and a digital ticket with a unique QR code is generated.
- **Main Success Scenario (Basic Flow):**
  1. Student navigates to the Event Discovery page and selects an event.
  2. System displays detailed event specifications (date, time, available seats).
  3. Student clicks the (Register Now) button.
  4. System verifies seat availability in real-time.
  5. System records registration and links it to the Student ID.
  6. System generates the digital ticket.
  7. System displays a success confirmation modal with the QR ticket.
- **Extensions (Alternative Flows):**
  - *4a. Event Reaches Full Capacity (Capacity = 0):* System disables registration, informs the student, and offers the option to join the waitlist.
  - *4b. Already Registered:* System detects an existing active ticket for this student and displays a warning:(You are already registered.)
- **Includes / Extends (Relationships):**
  - **Include:** *Generate Digital Ticket* (Step 6).
  - **Extend:** *Join Waitlist* (Step 4a).

- **Special Requirements:** The system must handle concurrent registration requests using database transaction locks to prevent overbooking.
- **Assumptions:** The student has a stable internet connection during the transaction.
- **Frequency of Use:** High, especially during the first 24 hours of an event's announcement.

UC-02: Scan QR Ticket for Attendance

- **Use Case ID:** UC-02
- **Use Case Name:** Scan QR Ticket for Attendance
- **Primary Actor:** Club Leader
- **Stakeholders and Interests:**
  - *Club Leader:* Wants a fast, error-free way to admit students at the door.
  - *Student:* Wants quick access to the event without waiting in long queues.
- **Preconditions:** Club Leader is logged into the scanner interface and has granted camera permissions.
- **Postconditions:** Student's attendance status is updated to(Attended) in the database, with a timestamp logged.
- **Main Success Scenario (Basic Flow):**
  1. Club Leader opens the scanner view.
  2. Leader scans the QR code presented on the student's mobile device.
  3. System decrypts the payload and validates the ticket string.
  4. System confirms ticket validity and checks that attendance status is(Pending).
  5. System updates student status to(Attended) and logs check-in timestamp.
  6. System displays a visual green success notification.
- **Extensions (Alternative Flows):**
  - *2a. Unreadable/Damaged Screen:* Leader switches to manual mode and searches the student by university ID.
  - *4a. Duplicate Scan (Ticket Already Used):* System detects status is(Attended), flags a red warning icon, and plays an error sound.
  - *4b. Invalid/Fraudulent Ticket:* System fails to decrypt payload and displays:(Invalid Ticket.)
- **Includes / Extends (Relationships):**

  - **Include:** *Validate Ticket Payload* (Step 3).
  - **Extend:** *Manual Attendance Check-in* (Step 2a).
- **Special Requirements:** The scanning and validation process must complete in under 4 seconds to prevent bottlenecks at the entrance.
- **Assumptions:** The venue has a working internet connection (Wi-Fi or cellular data) for real-time validation.
- **Frequency of Use:** Very high during the 30-minute window before an event starts.

UC-03: Submit Club Expense Request

- **Use Case ID:** UC-03
- **Use Case Name:** Submit Club Expense Request
- **Primary Actor:** Club Leader
- **Stakeholders and Interests:**
  - *Club Leader:* Wants to easily log expenses and get budget approvals.
  - *System Admin (Student Affairs):* Wants clear, trackable financial records with valid receipt proofs.
- **Preconditions:** The club has an active allocated semester budget, and the Club Leader is logged in.
- **Postconditions:** An expense entry is logged in(Pending Audit) status, awaiting administrative review.
- **Main Success Scenario (Basic Flow):**
  1. Club Leader navigates to the Financial Management module.
  2. Leader inputs expense details (Amount, Category, Date, Purpose).
  3. Leader uploads an image file (PNG/PDF) of the original receipt.
  4. System validates that the amount does not exceed the remaining budget balance.
  5. Leader submits the request.
  6. System saves the transaction record and sends an automated review alert to Student Affairs.
- **Extensions (Alternative Flows):**
  - *3a. Invalid File Format/Size:* System displays an error:(Receipt must be JPG/PNG/PDF under 5MB.)

- *4a. Insufficient Budget Balance:* System blocks submission and displays:(Expense exceeds available budget.)

- **Includes / Extends (Relationships):**
  - **Include:** *Upload Receipt Image* (Step 3).
- **Special Requirements:** Uploaded receipt files must be securely stored and restricted to authorized admins and the respective club leaders only.
- **Assumptions:** The Club Leader has access to a clear, legible digital copy of the physical receipt.
- **Frequency of Use:** Moderate; typically used a few times a month per active club.

3.3 Domain Model

The domain model captures the key conceptual entities, their attributes, and their static structural relationships within the Campus Event & Student Club Hub system.

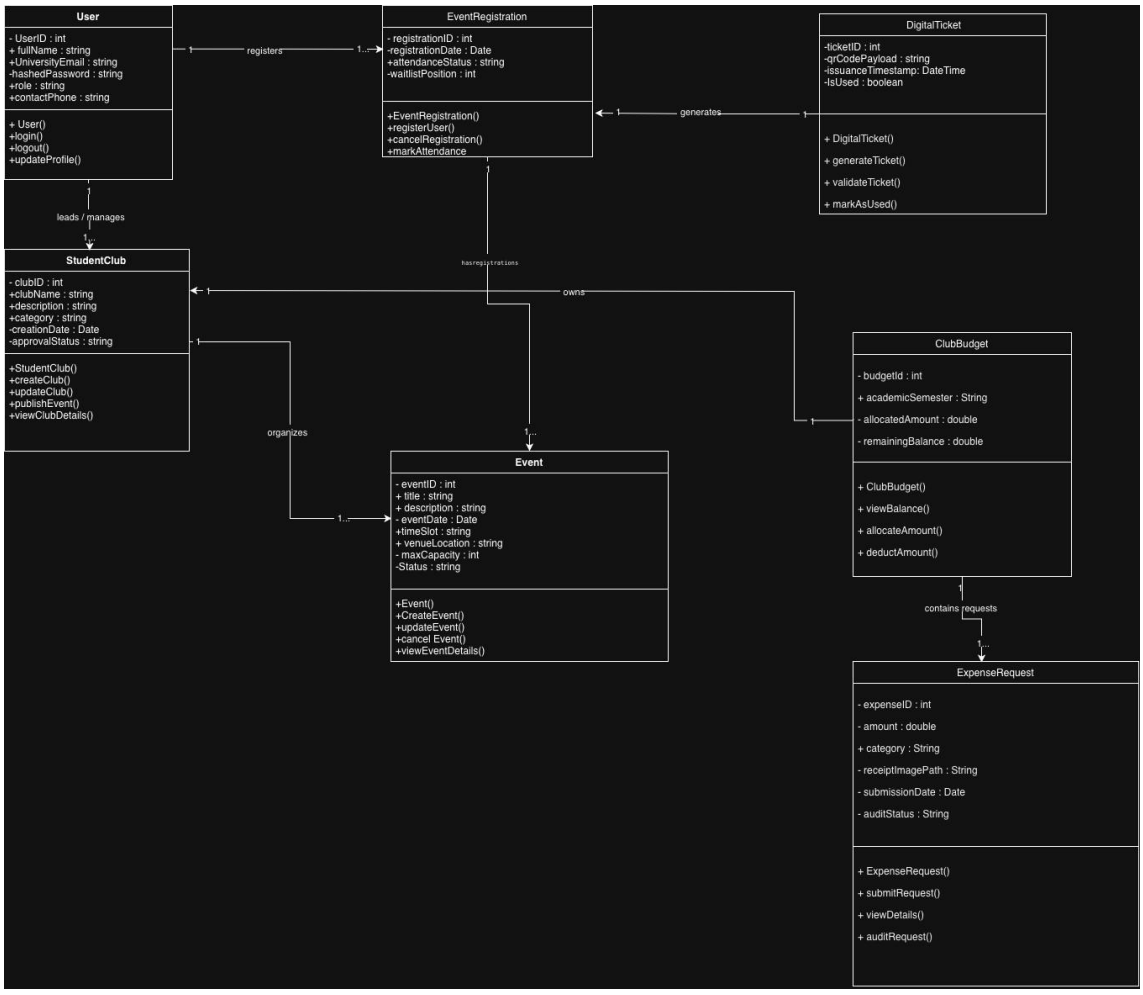


Figure 4: Conceptual Domain Class Diagram showing core entities and multiplicity associations.

3.3.1 Conceptual Class Diagram

Figure 4 depicts the core domain entities, illustrating ownership, association, and multiplicity constraints governing system state logic.

3.3.2 Class Descriptions

Below is the structural breakdown of the domain classes identified in the conceptual model.

1. User Class

- **Purpose:** Represents any authenticated individual accessing the platform (Students, Club Leaders, or System Administrators).
- **Attributes:** userID, fullName, universityEmail, hashedPassword, role, contactPhone.
- **Associations:**
  - Has a $1 \ldots *$ (one-to-many) relationship with **EventRegistration** (a student can register for multiple events).
  - Has a $0 \ldots 1$ optional association with **StudentClub** as a Club Leader.

2. StudentClub Class

- **Purpose:** Encapsulates a recognized student organization operating on campus.
- **Attributes:** clubID, clubName, description, category, creationDate, approvalStatus.
- **Associations:**
  - Has a $1 \ldots *$ relationship with **Event** (a club can organize and publish multiple campus events).
  - Has a $1 \ldots 1$ composition relationship with **ClubBudget** (each club possesses exactly one active budget ledger).

3. Event Class

- **Purpose:** Represents a scheduled campus event hosted by a student club.
- **Attributes:** eventID, title, description, eventDate, timeSlot, venueLocation, maxCapacity, status.
- **Associations:**
  - Associated with **StudentClub** ($* \ldots 1$, many events belong to one club).
  - Has a $1 \ldots *$ relationship with **EventRegistration** (an event contains multiple student registration records).

4. EventRegistration Class

- **Purpose:** Tracks an individual student's booking for a specific event.
- **Attributes:** registrationID, registrationDate, attendanceStatus, waitlistPosition.
- **Associations:**
  - Links exactly 1 **User** (1...1) to 1 **Event** (1...1).
  - Has a 1...1 composition relationship with **DigitalTicket**.

5. DigitalTicket Class

- **Purpose:** Represents the scannable electronic pass issued to an event attendee.
- **Attributes:** ticketID, qrCodePayload, issuanceTimestamp, isUsed.
- **Associations:**
  - Strictly bound (1...1) to a single **EventRegistration** record.

6. ClubBudget Class

- **Purpose:** Manages the financial ledger, tracking allocated funds, and remaining balance for a student club.
- **Attributes:** budgetId, academicSemester, allocatedAmount, remainingBalance.
- **Associations:**
  - Bound (1...1) to a **StudentClub**.
  - Has a 1...∗ relationship with **ExpenseRequest**.

7. ExpenseRequest Class

- **Purpose:** Represents an expense item logged by a club leader requiring financial auditing.
- **Attributes:** expenseID, amount, category, receiptImagePath, submissionDate, auditStatus.
- **Associations:**
  - Associated (∗...1) with a **ClubBudget**.

3.4 Non-Functional Requirements

This section outlines the 5 non-functional quality attributes of the system. Each requirement is accompanied by a specific testing plan and success criteria to allow empirical verification during Phase 4 (Implementation & Testing).

3.4.1 NFR-01: Performance — Response Time

Description: The platform shall provide low-latency responses under normal operating loads to ensure a smooth user experience.

Testing Plan: Execute automated load testing using Apache JMeter to simulate 100 concurrent user requests hitting the event browsing page (/events) simultaneously.

Success Criteria: The system must maintain an average HTTP response time of less than 5**.0 seconds** for 95% of requests, with zero server timeout errors.

3.4.2 NFR-02: Security — Password Hashing & Role Protection

Description: All sensitive user credentials must be stored securely, and administrative end-points must strictly enforce role authorization.

Testing Plan: Conduct static code analysis and manual penetration testing. Attempt to access administrative routes (e.g., '/admin/budgets') directly via unauthorized HTTP GET requests from a standard student session. Verify password storage in the database.

Success Criteria: All passwords in the database must be hashed using bcrypt (or Argon2) with salt. Direct URL manipulation attempts by standard students must yield an HTTP 403 Forbidden error page.

3.4.3 NFR-03: Usability — Responsive Design & Accessibility

Description: The web application interface must adapt dynamically to various screen resolutions, providing full usability on both mobile smartphones and desktop devices.

Testing Plan: Test the application user interface across multiple viewports (Desktop 1920 × 1080, Mobile 375 × 667) using Google Chrome DevTools and Google Lighthouse automated auditing tools.

Success Criteria: The interface must render without horizontal scrollbars on mobile viewports, and the automated Google Lighthouse Usability/Accessibility score must be at least **85/100**.

3.4.4 NFR-04: Reliability — Data Consistency & Transaction Integrity

Description: The system must maintain database state consistency during concurrent operations, preventing race conditions like double booking or duplicate ticket issuance.

Testing Plan: Execute a concurrency test script where 10 automated client threads attempt to register simultaneously for an event that has only 1 remaining seat.

Success Criteria: Exactly 1 thread must successfully register and receive a ticket, while the remaining 9 threads receive a(Capacity Full) notification. The event attendance count must not exceed maximum capacity.

3.4.5 NFR-05: Maintainability — Architectural Conformance

Description: The software source code must strictly adhere to the Model-View-Controller (MVC) architectural pattern to facilitate modularity and future maintenance.

Testing Plan: Perform an architectural code audit during code reviews. Verify that database queries are isolated within Model classes, route handling within Controllers, and UI logic within View templates.

Success Criteria: 100% of controller actions must be free of raw SQL queries, and View templates must not contain business logic or direct database connection calls.

3.5 External Interface Requirements

This section outlines the interface characteristics and protocols governing the platform's interactions with human users, hardware devices, external software components, and communication networks.

3.5.1 User Interface

The front-end user interface will be built as a responsive web dashboard adhering to clean visual hierarchy and accessibility standards.

- **Design & Layout:** Built using dynamic HTML5, CSS3, and modern UI components (e.g., Bootstrap/Tailwind). The design utilizes a consistent color palette aligned with university branding.
- **Responsiveness:** Layouts automatically adjust across Desktop (1920 × 1080), Tablet (768 × 1024), and Mobile (375 × 667) screen resolutions.
- **Navigation & Feedback:** Includes a persistent top navigation bar, intuitive breadcrumbs, real-time toast notifications for system alerts, and clear confirmation modals for sensitive actions (e.g., event deletion).

3.5.2 Hardware Interface

As a web-based application hosted on server infrastructure, direct hardware dependencies are minimized; however, specific client-side hardware integrations are required:

- **Camera / Optical Scanner:** The mobile browser interface requires HTML5 MediaDevices API access to utilize device cameras (smartphones or webcams) for real-time QR code scanning at event venues.
- **Display Devices:** Renderable across standard consumer monitors, laptop displays, and smartphone screens with touchscreen capabilities.

3.5.3 Software Interface

The software platform communicates with external software packages, underlying operating systems, and database management systems:

- **Database Management System (DBMS):** Connects to a relational database (e.g., MySQL or PostgreSQL) via standard Object-Relational Mapping (ORM) or secure database drivers for data persistence.
- **Email Gateway Service:** Integrates with standard SMTP server protocols (e.g., SendGrid or University SMTP) to dispatch automated transactional emails (digital tickets, event cancellation alerts).
- **Web Server & Operating System:** Deployed on Linux-based environments (e.g., Ubuntu Server) running web server software (e.g., Nginx / Apache).

3.5.4 Communication Interface

All client-server data transmissions are governed by standardized secure communication proto cols:

- **HTTPS Protocol:** All web HTTP traffic is strictly encrypted using Hypertext Transfer Protocol Secure (HTTPS) over TLS 1.3 encryption (Port 443) to guarantee data privacy and prevent eavesdropping.
- **Data Exchange Standards:** Asynchronous client-server communications (AJAX/Fetch API) pass structured payload data serialized in JSON format.

A Appendices

A.1 Appendix A: Data Dictionary

This appendix provides a high-level data dictionary outlining the primary database entities, table names, and structural descriptions planned for the system implementation.

A.2 Appendix B: Glossary

- **Digital Ticket:** An electronic pass, often featuring a unique barcode or QR code, generated by the system to verify a student's registration for an event.
- **Event Capacity:** The maximum number of students allowed to register for a specific event.
- **Budget Allocation:** The initial sum of money provided to a student club by the university for a given academic semester.
- **MVC (Model-View-Controller):** A software design pattern that separates internal representations of information from the ways information is presented to and accepted from the user.

Table Name	Primary Key	Description / Purpose
users	user_id	Stores user account records, credentials, profile details, and system roles (Student, Leader, Admin).
clubs	club_id	Contains student organization metadata, creation details, and approval statuses.
events	event_id	Holds event parameters, scheduling details, venue locations, and seat capacities.
registrations	registration_id	Records student event reservations, check-in status, and waitlist tracking.
tickets	ticket_id	Stores generated digital QR ticket payloads, timestamps, and usage flags.
budgets	budget_id	Tracks allocated club semester finances, expenditure totals, and balance limits.
expenses	expense_id	Logs individual financial receipt entries, expense amounts, and admin audit statuses.

Table 1: Data Dictionary summarizing core database tables.